

O SOLVER **fmincon** NO MATLAB

OTIMIZAÇÃO CONTÍNUA DETERMINÍSTICA

Alunos: Leonardo Ronald Perin Rauta

Maurício da Silva Justino

Romulo de Aguiar Beninca

Professor: Prof. Dr. Carlos Alberto Bavastri

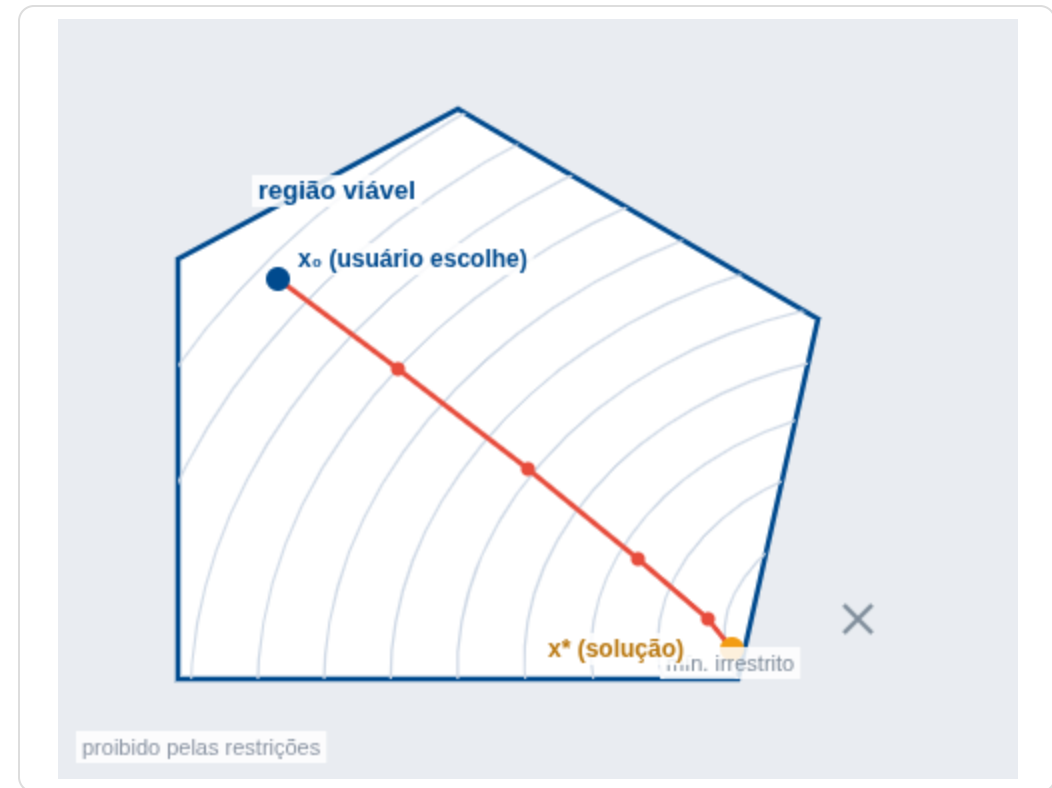
UFPR - Engenharia Mecânica

O QUE É O `fmincon`?

Function MINimization with CONstraints — função do pacote *Optimization Tools* que tem como objetivo minimizar uma função não linear *sujeita a restrições*.

$$\begin{aligned} \min_x f(x) \text{ sujeito a:} \\ Ax \leq b \cdot A_{eq} x = b_{eq} \\ c(x) \leq 0 \cdot c_{eq}(x) = 0 \\ lb \leq x \leq ub \end{aligned}$$

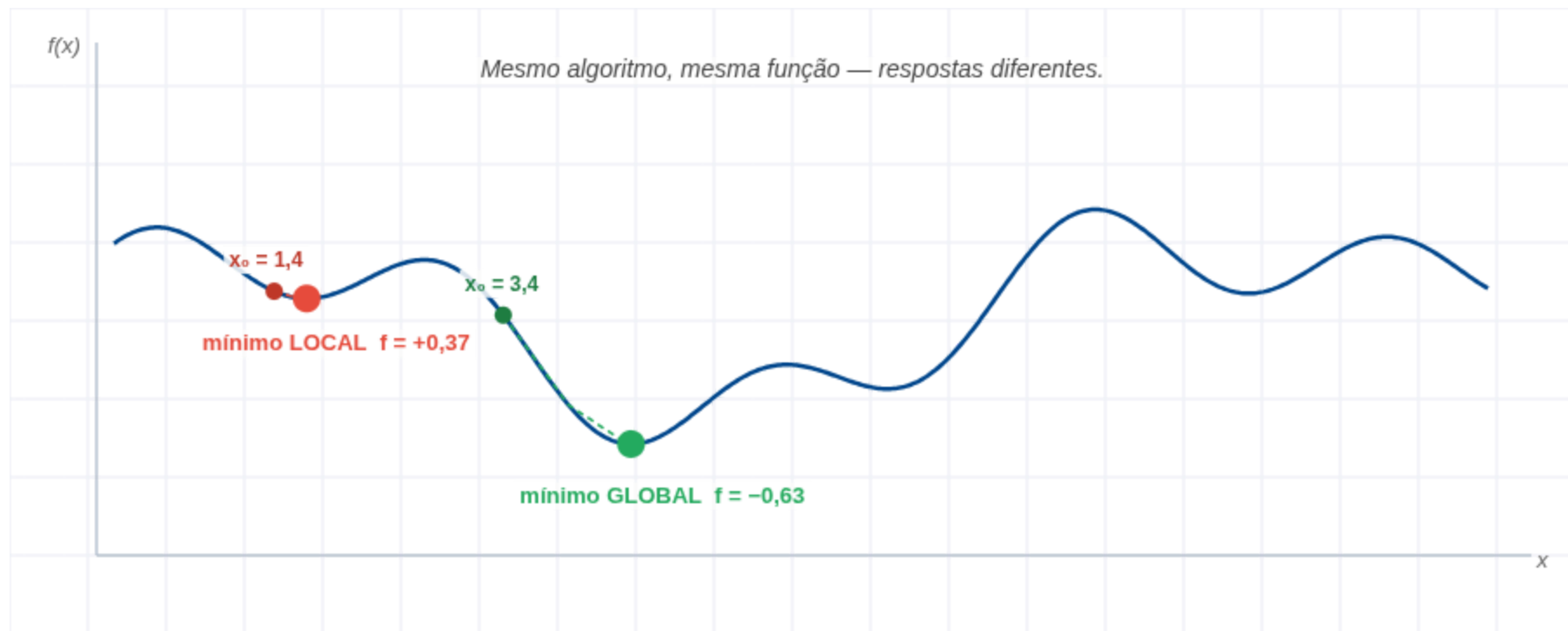
- **Busca local:** anda "morro abaixo" usando **derivadas (gradiente)**.
- **Depende do ponto inicial** x_0 — quem escolhe é o usuário.
- Acha o **mínimo local** onde x_0 **caiu**, não garante o mínimo global.



A região branca é a **zona viável**; o cinza é proibido pelas restrições.

POR QUE x_0 IMPORTA TANTO?

O `fmincon` é **determinístico e local**: dado o mesmo x_0 , ele sempre desce pelo mesmo caminho. Dois pontos de partida diferentes podem levar a **dois mínimos diferentes**.



Consequência prática: em problemas multimodais, roda-se o `fmincon` a partir de **vários** x_0 (*multistart*) e fica-se com o melhor resultado.

EVOLUÇÃO DOS MÉTODOS CLÁSSICOS

O que estudamos em sala continua lá dentro — mas o MATLAB **filtra** o que é eficiente:

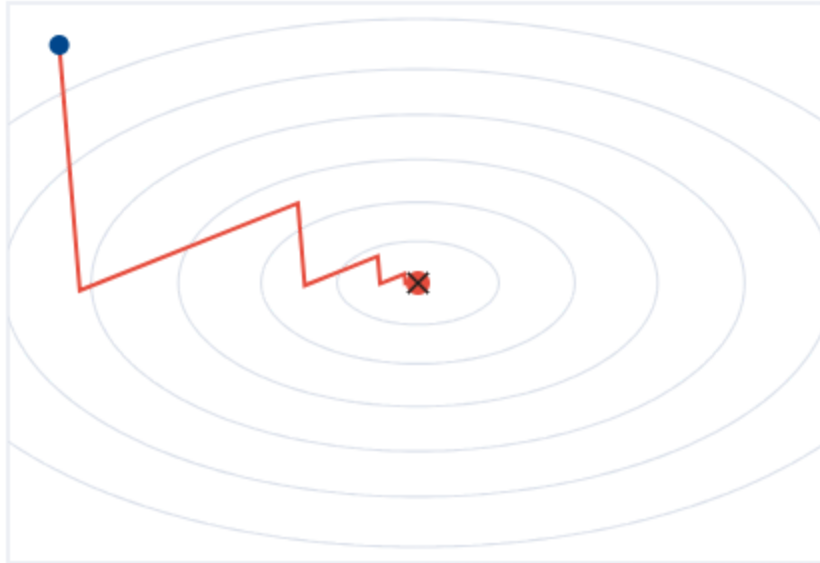
Método clássico	Destino no fmincon	Por quê
Dicotomia / Seção Áurea / Bolzano	Descartado	Na busca do passo λ^* usa-se interpolação polinomial com derivadas: bem mais rápida.
Gradiente nulo $\nabla f(x) = 0$	Reaproveitado	Não serve como busca geométrica, mas vira o critério de parada (condições de KKT).
Descida Mais Rápida (<i>Steepest Descent</i>)	Descartado	Ineficiente: faz ziguezague em vales estreitos (mal condicionados).
Aproximação Quadrática (Newton / Quase-Newton)	Motor principal	Usa inclinação + curvatura → convergência superlinear.

KKT (Karush-Kuhn-Tucker): conjunto de equações e desigualdades que caracterizam a solução ótima de um problema com restrições.

ZIGUEZAGUE VS. CURVATURA

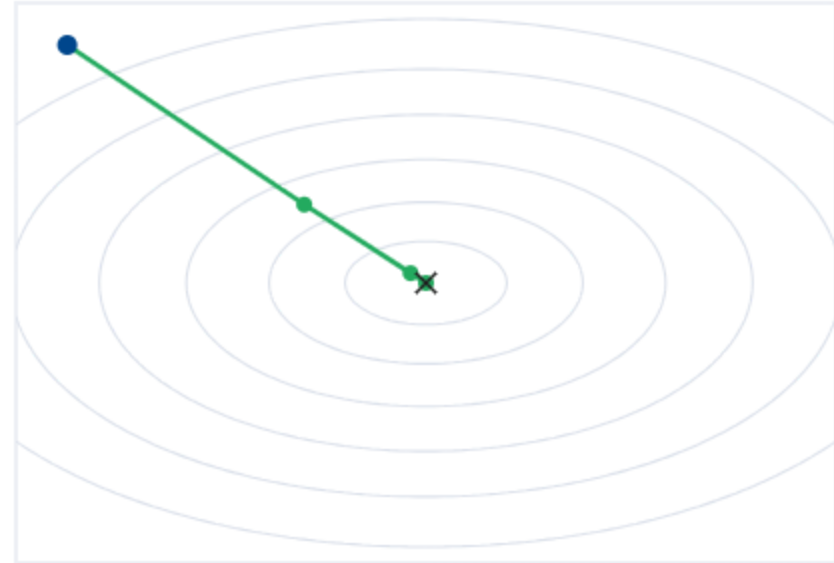
Num vale estreito, o gradiente aponta para a **parede**, não para o fundo. A **aproximação quadrática** corrige isso usando a curvatura (Hessiana):

Descida Mais Rápida — ziguezague



40 iterações

Quase-Newton (BFGS) — usa curvatura



3 iterações

Steepest Descent: $d = -\nabla f(x) \rightarrow$ centenas de passos curtos.

Quase-Newton (BFGS): $d = -H^{-1}\nabla f(x) \rightarrow$ poucos passos, direto ao fundo. (H = aproximação da Hessiana, construída iterativamente sem calcular derivadas segundas.)

ALGORITMOS INTERNOS

O `fmincon` não é *um* algoritmo — é uma **caixa com vários motores**. Você escolhe pelo `Algorithm`:

INTERIOR-POINT

(PADRÃO)

Usa **funções de barreira**: penalidade que tende ao infinito nas fronteiras, forçando o caminho a passar **por dentro** da zona viável. Ideal para larga escala.

SQP

Sequential Quadratic Programming (Programação Quadrática Sequencial).
Aproxima f por **parábolas** e as restrições por retas, resolvendo um subproblema por iteração (BFGS). **Rigoroso com as restrições**.

TRUST-REGION-REFLECTIVE

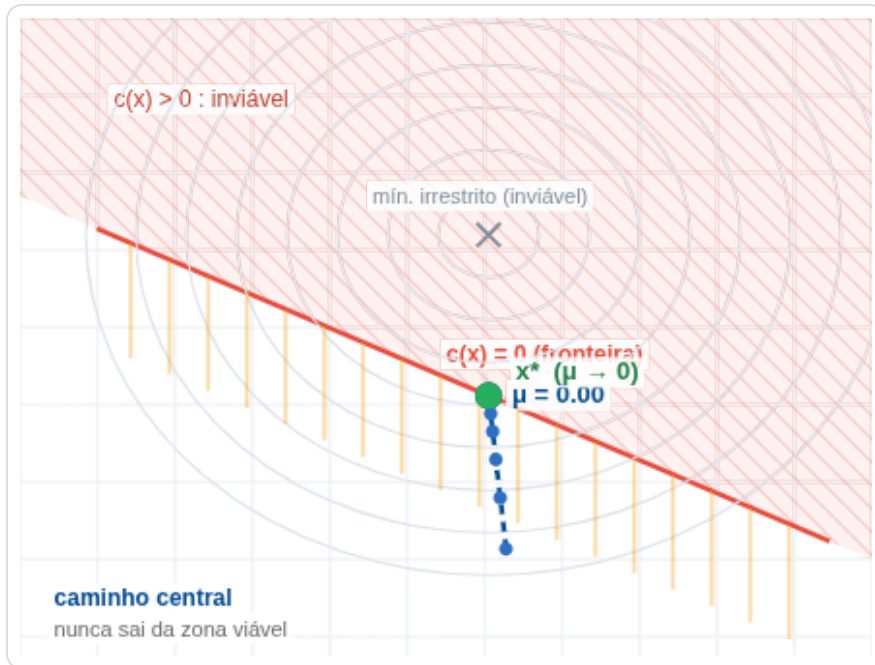
Cria um **raio de confiança** onde a série de Taylor ainda vale. Muito rápido, mas **não aceita restrições não lineares** e exige gradiente analítico.

BFGS = Broyden, Fletcher, Goldfarb, Shanno — atualização iterativa da Hessiana aproximada.

INTERIOR-POINT: A BARREIRA

A ideia: em vez de *verificar* a restrição a cada passo, ela é **incorporada à função objetivo** como penalização — a **barreira**. O problema restrito vira uma sequência de problemas **sem restrição**:

$$\min_x \underbrace{f(x)}_{\text{objetivo}} - \underbrace{\mu \sum_i \ln(-c_i(x))}_{\text{barreira (penalização)}}$$

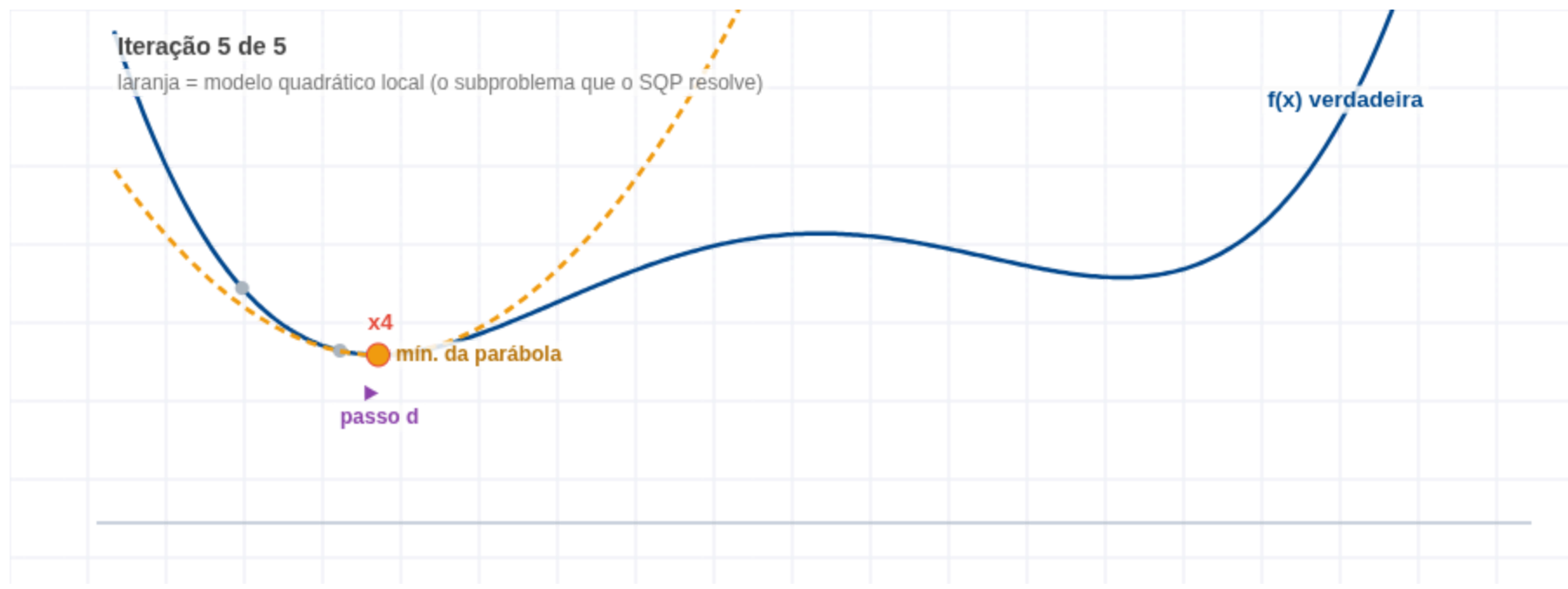


Na figura: o ponto azul desce pelo **caminho central**, sempre por dentro da zona viável. A cada etapa μ diminui, a barreira (laranja) afina e o ponto **encosta na fronteira vermelha** — até parar em x^* .

SQP: PARÁBOLA A CADA PASSO

SQP = Sequential Quadratic Programming — *Programação Quadrática Sequencial*. A sigla descreve o método: resolve uma **sequência** de subproblemas **quadráticos**.

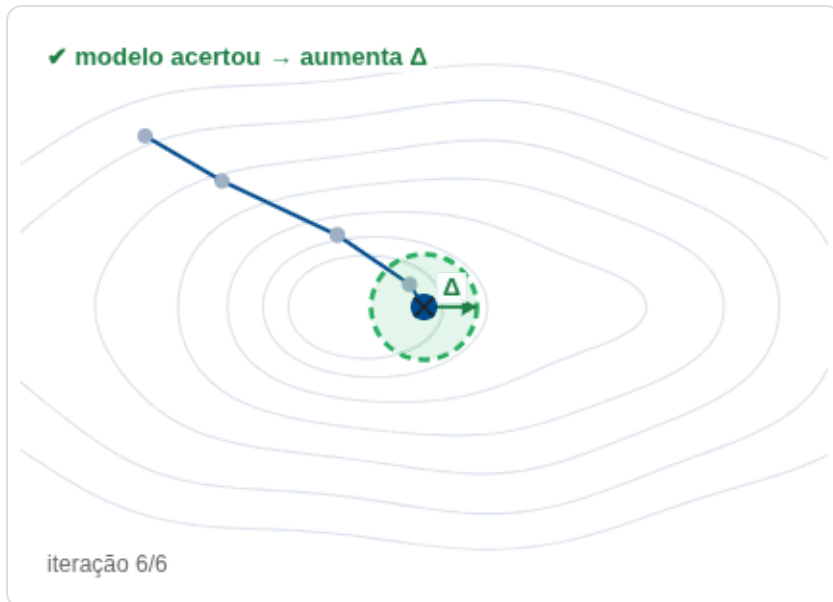
A cada iteração o 'sqp' troca o problema difícil por um **problema fácil e local**: aproxima f por uma **parábola** e as restrições por **retas**, resolve esse subproblema, dá o passo, e repete.



A parábola (laranja) toca a curva no ponto atual com a mesma inclinação e curvatura. O **mínimo da parábola** vira o próximo ponto — daí a convergência rápida perto da solução.

TRUST-REGION: ATÉ ONDE CONFIO?

A cada iteração, o algoritmo substitui $f(x)$ por uma **aproximação quadrática local**. Ela só é confiável **perto** de x_k — por isso o passo p fica limitado a uma região de raio Δ_k :



O círculo é Δ_k · verde aceito · vermelho rejeitado

$$\|p\| \leq \Delta_k$$

Depois do passo, o algoritmo **compara o que foi prometido com o que aconteceu**:

$$\rho_k = \frac{\text{melhora real em } f}{\text{melhora prevista pelo modelo}}$$

ρ_k	Leitura	Ação
≈ 1	Modelo previu bem	Aceita o passo e pode aumentar Δ_k
pequeno	Modelo pouco preciso	Reduz Δ_k
< 0	A função <i>piorou</i>	Rejeita o passo e encolhe a região

O raio Δ_k representa **até onde o algoritmo confia no seu modelo quadrático** — e é reajustado a cada iteração.

SINTAXE E UTILIZAÇÃO

Da chamada mínima ao problema totalmente restrito — tudo na mesma função:

```
% 1) Mínimo absoluto: função + ponto inicial
x = fmincon(fun, x0);

% 2) Com restrições lineares de desigualdade  $A*x \leq b$ 
x = fmincon(fun, x0, A, b);

% 3) Chamada completa (ordem fixa dos argumentos)
[x, fval, exitflag, output, lambda] = ...
    fmincon(fun, x0, A, b, Aeq, beq, lb, ub, nonlcon, options);

% 4) Ignorando restrições que o problema não tem: colchetes vazios
x = fmincon(fun, x0, [], [], [], [], lb, ub, @nolinear);

% 5) Forma moderna: uma única estrutura encapsulada
x = fmincon(problem);
```

 **A ordem posicional importa:** não se pode "pular" um argumento — passa-se [] no lugar dele.

PARÂMETROS DE ENTRADA

Variável	Significado no MATLAB	Equivalência matemática
<code>fun</code>	Função objetivo a minimizar (<i>function handle</i>)	$f(x)$
<code>x0</code>	Ponto de partida inicial (vetor)	x_0
<code>A, b</code>	Matriz e vetor de desigualdade linear	$Ax \leq b$
<code>Aeq, beq</code>	Matriz e vetor de igualdade linear	$A_{eq}x = b_{eq}$
<code>lb, ub</code>	Limites inferior (<i>lower</i>) e superior (<i>upper</i>)	$lb \leq x \leq ub$
<code>nonlcon</code>	Função com restrições não lineares ; retorna <code>[c, ceq]</code>	$c(x) \leq 0, c_{eq}(x) = 0$
<code>options</code>	Opções criadas com <code>optimoptions</code>	—

Atenção ao sinal: o MATLAB exige tudo na forma " ≤ 0 ". Uma restrição $x_1^2 + x_2^2 \geq 4$ deve ser reescrita como $4 - x_1^2 - x_2^2 \leq 0$ dentro do `nonlcon`.

PARÂMETROS DE SAÍDA

	Variável	O que é	Para que serve na prática
A SOLUÇÃO	x	O ponto ótimo encontrado (vetor de projeto).	É a resposta: os valores das variáveis que você vai usar.
	$fval$	O valor de f nesse ponto.	O desempenho atingido — massa, custo, energia.
	$exitflag$	Por que o algoritmo parou.	1 convergiu · 0 acabaram as iterações · -2 inviável. Parar não é convergir.
SOBRE A SOLUÇÃO	$output$	Diagnóstico: iterações, avaliações de f , algoritmo.	Mostra o <i>custo</i> da busca. Muitas avaliações → forneça o gradiente.
	λ	Estrutura com multiplicadores de lagrange, Um número por restrição : o quanto ela influenciou a solução.	Diz quais restrições prendem o projeto (veja abaixo).
	$grad / hessian$	Inclinação e curvatura de f na solução.	$\nabla f \approx 0$ confirma o ótimo; a curvatura diz se ele é agudo ou plano .

CONFIGURAÇÃO: `optimoptions`

O antigo vetor `foptions` foi descontinuado. Hoje o controle é feito por um objeto de opções:

```
options = optimoptions('fmincon', ...  
    'Algorithm',          'sqp', ...      % qual motor rodar  
    'SpecifyObjectiveGradient', true, ... % eu forneço o gradiente analítico  
    'Display',            'iter', ...     % imprime cada iteração  
    'StepTolerance',      1e-10, ...      % tolerâncias de parada  
    'OptimalityTolerance', 1e-8);
```

ALGORITHM

Escolhe o motor:

`'interior-point'`,
`'sqp'`, `'active-set'`,
`'trust-region-reflective'`.

SPECIFYOBJECTIVEGRADIENT

Desliga a **diferença finita** quando fornecemos ∇f analítico. Economiza n avaliações de f por iteração.

DISPLAY / TOLERANCES

`Display` controla o que aparece no terminal; as tolerâncias definem quando parar.

CRITÉRIOS DE PARADA E TOLERÂNCIAS

Quando o algoritmo decide que chegou? Três perguntas independentes — basta uma responder "sim":

STEPTOLERANCE

O passo geométrico ficou irrisório: $\|x_{k+1} - x_k\| < \varepsilon$.
"Parei de andar."

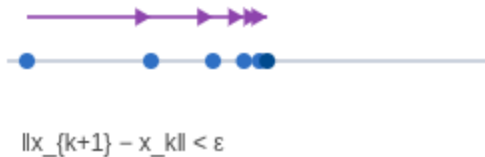
FUNCTIONTOLERANCE

A mudança de posição gera **variação vertical** desprezível:
 $|f_{k+1} - f_k| < \varepsilon$.
"A curva achatou."

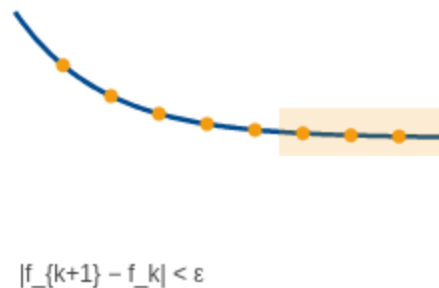
OPTIMALITYTOLERANCE

A norma do gradiente (projetado) é ~zero: as condições de 1ª ordem de KKT foram satisfeitas.
"Matematicamente é o ótimo."

Step: o passo sumiu



Function: a curva achatou



Optimality: $\nabla f \approx 0$



⚠ Só a **OptimalityTolerance** garante otimalidade. As outras duas podem disparar por **estagnação** — passo pequeno não é o mesmo que ótimo encontrado. Sempre confira o `exitflag`.

EXEMPLO COMPLETO

Minimizar a função de Rosenbrock dentro de um disco de raio 1:

$$\begin{aligned} \min f(x) &= 100(x_2 - x_1^2)^2 + (1 - x_1)^2 \\ \text{sujeito a } x_1^2 + x_2^2 &\leq 1 \end{aligned}$$

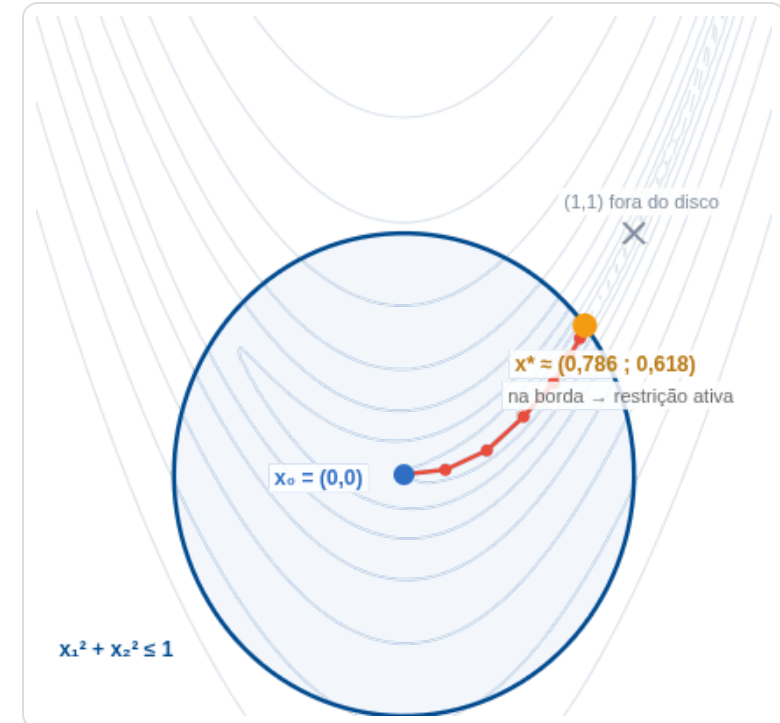
```
% função objetivo (Rosenbrock)
fun = @(x) 100*(x(2)-x(1)^2)^2 + (1-x(1))^2;

% restrição não linear: c(x) <= 0 → disco unitário
function [c, ceq] = disco(x)
    c = x(1)^2 + x(2)^2 - 1; % x1^2+x2^2-1 <= 0
    ceq = []; % sem igualdade
end

x0 = [0, 0];
opt = optimoptions('fmincon','Algorithm','sqp','Display','iter')

[x, fval, exitflag, ~, lambda] = ...
    fmincon(fun, x0, [], [], [], [], [], [], [], @disco, opt);
```

Resultado: $x^* \approx (0,7864; 0,6177)$ · $f_{val} \approx 0,0457$
 $\text{exitflag} = 1$ (convergiu) · $\lambda_{ineq} > 0 \rightarrow$ restrição ativa.



O ótimo irrestrito é $(1, 1)$ — fora do disco. A solução para na borda.

CONCLUSÃO

O principal benefício do `fmincon` está na sua **arquitetura modular**: um mesmo comando cobre desde uma minimização simples até sistemas altamente restritos.



ALGORITMOS CONSOLIDADOS

Pontos interiores, SQP e estimativas Quase-Newton (BFGS) reunidos em uma só interface.



CONTROLE PARAMÉTRICO

O `optimoptions` dá controle total de tolerâncias, gradientes e diagnóstico.



EFICIÊNCIA

Processamento eficiente de sistemas não lineares restritos de engenharia.

REFERÊNCIAS

- THE MATHWORKS. *fmincon* — *Find minimum of constrained nonlinear multivariable function*. Optimization Toolbox Documentation. Disponível em: mathworks.com/help/optim/ug/fmincon.html.
- THE MATHWORKS. *Choosing the Algorithm e Constrained Nonlinear Optimization Algorithms*. Optimization Toolbox User's Guide.

DÚVIDAS?

Agradecemos pela atenção!

**Leonardo Ronald Perin Rauta · Maurício da Silva Justino ·
Romulo de Aguiar Beninca**

Prof. Dr. Carlos Alberto Bavastri · UFPR — Engenharia Mecânica

